From Data to Solutions · Weekend 7

# Fine-tuning methods for LLMs

Adapting a model you did not train

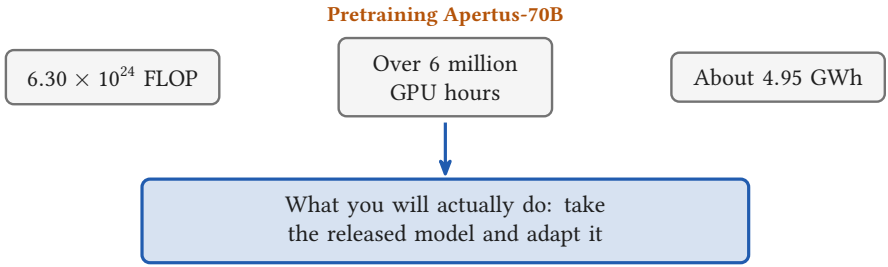
Carlos Cotrini

Section 1

# From pretraining to adapting

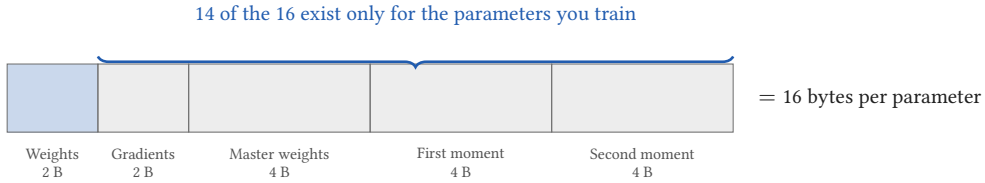
1

# You will start from a released model




- Two days have priced one pretraining run
- None of it is a decision you will ever make
- You cannot pretrain, so you adapt, and the same ledger prices that

# Sixteen bytes per parameter




- Mixed precision Adam, from this morning: $2 + 2 + 4 + 4 + 4$
- Apertus-8B needs 128.8 GB of state, and a GH200 holds 96 GB
- Freeze a parameter and 14 of its 16 bytes disappear

Section 2

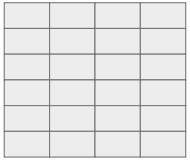
# One matrix, and a low rank update

2

One matrix, and a low rank update

# Twenty-four numbers, or ten

$W$: 6 outputs × 4 inputs



Frozen: no gradient, no moments

$B$: 6 × 1

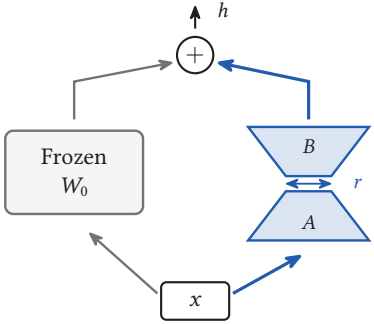
$A$: 1 × 4

| | $a_1$ | $a_2$ | $a_3$ | $a_4$ |
|---|---|---|---|---|
| $b_1$ | $b_1 a_1$ | $b_1 a_2$ | $b_1 a_3$ | $b_1 a_4$ |
| $b_2$ | $b_2 a_1$ | $b_2 a_2$ | $b_2 a_3$ | $b_2 a_4$ |
| $b_3$ | $b_3 a_1$ | $b_3 a_2$ | $b_3 a_3$ | $b_3 a_4$ |
| $b_4$ | $b_4 a_1$ | $b_4 a_2$ | $b_4 a_3$ | $b_4 a_4$ |
| $b_5$ | $b_5 a_1$ | $b_5 a_2$ | $b_5 a_3$ | $b_5 a_4$ |
| $b_6$ | $b_6 a_1$ | $b_6 a_2$ | $b_6 a_3$ | $b_6 a_4$ |

$\Delta W = BA$

- Full fine-tuning updates all 24
- LoRA freezes $W$ and learns $B$ and $A$: $6 + 4 = 10$ numbers, a ratio of 2.4
- Every row of $BA$ is the row $A$, rescaled. That is what rank one means

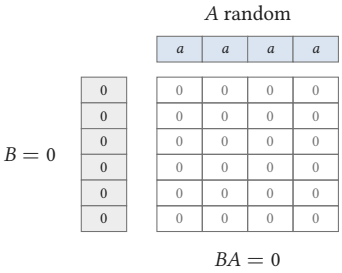
# The adapter is added, not substituted




$$h = \underbrace{W_0 x}_{\text{frozen}} + \underbrace{\frac{\alpha}{r} BAx}_{\text{trained}}$$

- Both paths read the same $x$, and their outputs are added
- $\alpha / r$ rescales the adapter, and it behaves like a learning rate
- The paper's own tables use ratios of 1, 2 and 8; the PEFT default is $r = 8$, $\alpha = 8$

# B starts at zero, A starts random

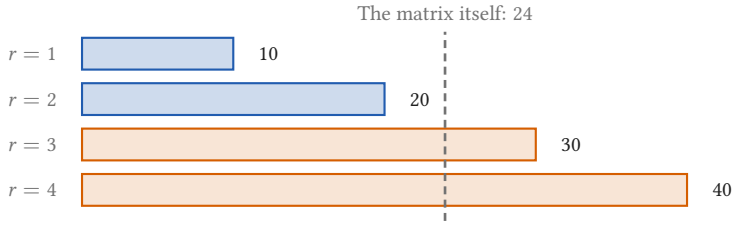



$$W_0 + BA = W_0$$

The first forward pass is exactly the pretrained model

- The adapter does nothing at all on step one
- So training starts where pretraining stopped, not somewhere random
- If both started at zero, both gradients would be zero and nothing would ever move

# Rank three costs more than the matrix




- Break even at $r = dk/(d + k) = 2.4$ on this matrix
- On a square $d \times d$ matrix that is $r = d/2$, and the saving is $d/(2r)$
- Llama-2-7B's query projection is $4096 \times 4096$: at $r = 8$ that is 65 536 numbers against 16 777 216, a factor of 256

Section 3

# The same picture at real scale

3

The same picture at real scale

# Rank 16, and 0.08 percent of the model trains

$8.05 \times 10^9$ parameters

6 815 744 trainable, 0.0846 percent of the model

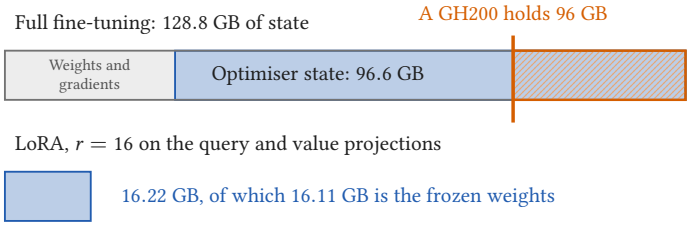
| Target modules | Rank | Trainable | Percent |
|---|---|---|---|
| `q_proj`, `v_proj` (the PEFT default) | 16 | 6 815 744 | 0.0846 |
| All six linear layers | 16 | 39 845 888 | 0.4948 |
| All six linear layers | 64 | 159 383 552 | 1.9792 |
| All six linear layers | 256 | 637 534 208 | 7.9167 |

- The lower bar is drawn to scale, which is why you cannot see it
- Even rank 256 on all six linear layers stays under 8 percent

Apertus-8B's shape is reconstructed from its published totals: 32 layers, $d_{\text{model}} = 4096$, 8 key and value heads.

The same picture at real scale

# 128.8 GB does not fit; 16.22 GB does



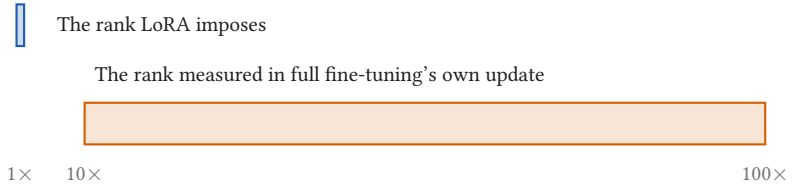

- Optimiser state: 96.6 GB becomes 0.082 GB
- Total state falls only 7.9 times, because the frozen weights stay
- The adapter file is 13.6 MB, against a 16.1 GB base model
- What needed more than one accelerator this morning now needs one

Section 4

# What the low rank assumption costs

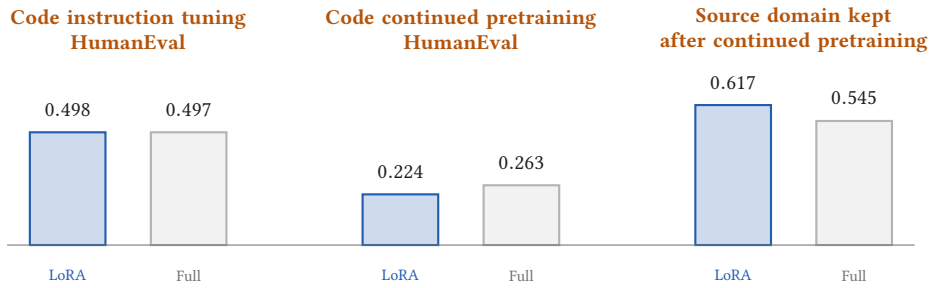
4

# Low rank is a hypothesis, not a theorem



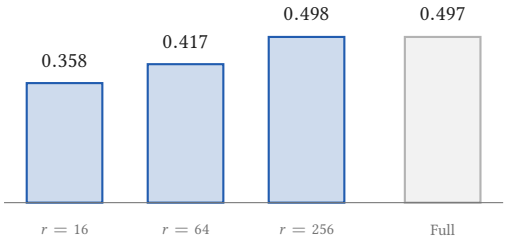

- Hu et al. wrote "we hypothesize the updates to the weights also have a low intrinsic rank"
- Aghajanyan measured a random subspace, not a weight update: $d_{90} = 207$ for RoBERTa-Large on MRPC
- Biderman measured the update itself: 10 to 100 times typical LoRA ranks, and higher in the MLP than in attention
- Low rank is a constraint you impose. It is often harmless, and nobody proved it has to hold

# Three comparisons with full fine-tuning

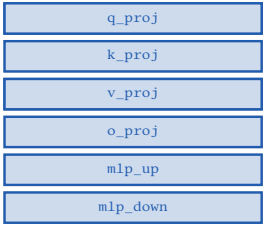



- Tulu 2, eight evaluations averaged: 47.7 against 53.0 at 7B, and 71.0 against 73.4 at 70B
- The gap is worst on open ended generation, and it shrinks with model size
- Higher is better in all three panels, so the third is the one LoRA wins

# Rank and target modules are real knobs




HumanEval, code instruction tuning




What every study since recommends

- `q_proj` and `v_proj` is PEFT's default, faithfully reproducing the 2021 paper
- QLoRA: adapters on all linear layers are required to match full fine-tuning
- Biderman: most of the gain comes from the MLP, not from attention

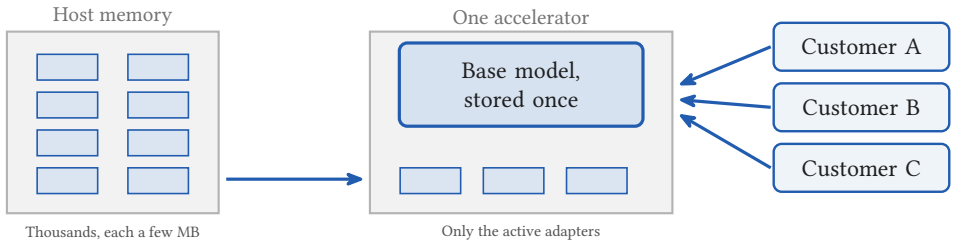
Section 5

# One base model, many adapters

5

One base model, many adapters

# Two thousand adapters on one machine




- 100 adapted copies of GPT-3: about 354 GB, against about 35 TB
- S-LoRA on one A100: 8.05 requests per second at 5 adapters, 7.61 at 2000
- The packed baseline manages 2.04 at 5 adapters and runs out of memory at 100

The catch: merging $W_0 + BA$ forces one copy of the weights per adapter, so you get the merge or the shared batch.

One base model, many adapters

# Four things this half hour did not cover

| Full fine-tuning mechanics | QLoRA: the base at 4 bits | Prefix and prompt tuning | Preference tuning and DPO |
|---|---|---|---|
| Priced by this morning's ledger | Apertus-8B: 16.1 GB becomes 4.15 GB | | Preference data is Weekend 2's material |

- Thirty minutes buys one mechanism, priced with the same ledger as the rest of the weekend
- LoRA is not a cheaper full fine-tune. It is a different point on a trade off, and often the point you want
- The 11:00 exercise is where you run it